Bil482 - Software Design Document: Wi$h Li$t

Contributors:

Akif Emre Reis
Alaaddin Gürsoy
Elif Serra Öncü
Gizem Elif Bayar

Department of Computer Engineering
TOBB University of Economics & Technology

Spring 2026

1. System Overview

The Wishlist and Price Tracking Application (Wi$h Li$t) is a comprehensive system designed to enable users to create, manage, and track products and their prices over time. It operates as a foundational organizational layer for categories while driving automated product scraping, price tracking, and notification capabilities. The module ensures that products are grouped logically, parsed dynamically through web scraping, and monitored continuously.

The module is implemented as a web-based client-server interaction where:

- The frontend (React) provides a rich user interaction layer, including Dashboards, intuitive Modals, and visual analytics (price charts).
- The backend (FastAPI with Python) handles business logic, task scheduling (APScheduler) for price tracking, and applies design patterns (Factory, Strategy, Observer) for extensibility.
- Supabase (PostgreSQL) stores the relational data including users, categories, products, price histories, and notifications.

2. System Context

Primary actor: User

Supporting Systems:

- Supabase
- FastAPI Backend (with APScheduler)
- React Frontend
- External E-Commerce Systems (Amazon, Trendyol, Hepsiburada, Boyner, etc.)

Context Description:
The User interacts with the React UI → React sends REST HTTP requests to FastAPI → FastAPI validates data, manages categories/products, and triggers Scraper Strategies → Background Scheduler periodically checks for prices from Third-Party domains → Supabase stores data. Notifications are triggered via the Observer pattern upon threshold events.

3. Key Features and Functionality

The system shall:

1. Create, edit, and delete custom categories.
2. Initialize default categories (Cosmetics, Clothing, Technology, Kitchen).
3. Allow users to add products with URL inputs; delegating metadata extraction to scrapers.
4. Support setting target prices, priority levels, check frequencies, and marking products as Favorites.
5. Schedule periodic price checks (hourly, daily, weekly) running automatically in the background.
6. Keep track of price changes and store them as a sequence for dynamic charting.
7. Alert the user (via Notifications) when a price experiences a change or reaches a defined target limit.

4. Assumptions and Dependencies

Assumptions

- Each category and product belongs to a single user.
- Category names must be unique per user.
- User authentication is handled through a temporary manual token/user ID configuration until full Auth integration.
- Target prices and favorite status can be updated dynamically at any time without resetting tracking history.

Dependencies

- Supabase PostgreSQL availability and Storage connection.
- Backend background runtime availability (APScheduler).
- E-Commerce site DOM structural consistency. To offset rigid structures, an LLM Scraper fallback scheme is integrated.

5. Architectural Design

5.1 System Architecture Diagram (High-Level)

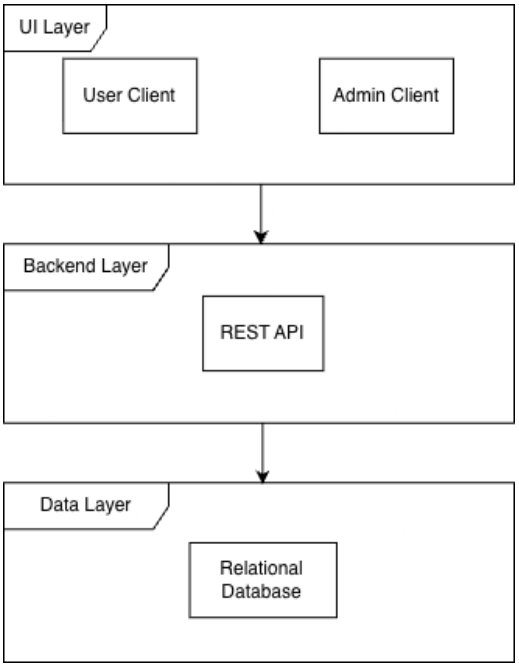


Figure 1. High Level Design

Frontend Layer: React (Component-Based UI)
Application Layer: FastAPI (Python) enclosing Routers, Business Logic Services, Factories, Observers, and Schedulers.
Data Layer: Supabase PostgreSQL.

5.2 Architectural Patterns and Styles

The system follows:

Client-Server Architecture:

Absolute decoupling of presentation and data/business logic.

Layered Architecture (Backend):

The backend is structured into layers:

- Presentation Layer (app/api/)
- Service Layer (app/services/)
- Persistence Layer (app/repositories/ + Supabase).

RESTful API Design:

Stateless communication.

Event-Driven / Background Task Architecture:

Utilizing APScheduler and Observer models independently from the HTTP event loop to manage async scraping.

5.3 Rationale for Architectural Decisions

- The layered design improves maintainability and strict separation of concerns.
- Extending Scrapers via Strategy and Factory Design Patterns allows adding new supported websites without impacting the core system.
- Supabase provides a fully managed PostgreSQL lowering operational complexities.

6. Component Design

6.1 Subsystem and Modules

Backend Components

1. Routers (app/api/): Category, Product, User
2. Services (app/services/): Handles business executions.
3. Repositories (app/repositories/): DB queries and persistence.
4. Factories (app/factories/): CategoryFactory, ScraperFactory, UserFactory
5. Scrapers (app/scrapers/): Strategy implementations (Amazon, Trendyol, Hepsiburada, LLM Fallback)
6. Observers (app/observers/): Price monitor and event distribution
7. Scheduler (app/scheduler/): Price loop triggering

Frontend Components

1. WishlistDashboard: Central interactive product grid with filtering.
2. AddCategoryModal & AddProductModal: Data entry and validation UI.
3. ProductDetailModal: Individual product views and price history visualization.
4. NotificationBell: In-app notification drop-downs.

6.2 Responsibilities of Each Component

- **Routers:** Validate request body (Pydantic), route paths, mapping correctly to correct service calls.
- **Services:** Inject application rules, validate business logic (e.g., verifying limits).
- **Factories / Scrapers:** Instantiate logic context based on domain (e.g., matching amazon.com.tr URL to AmazonScraper).

- **Observers:** Trigger decoupled notifications (e.g., TargetPriceNotificationObserver) upon detection of price drops in the background job without crowding the service logic.

6.3 Interfaces Between Components

Internal backend components communicate via direct method invocations following a layered architecture approach.

6.4 Component Diagrams

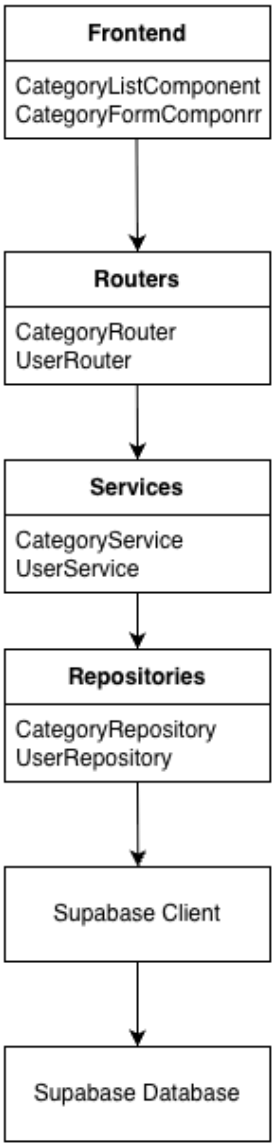


Figure 2. Component Diagram

1. Category Creation and Management

This process includes users creating their custom categories, listing existing ones, and generating predefined categories (Electronics, Clothing, etc.) when they first register. The CategoryFactory handles the instantiation of both predefined and custom categories.

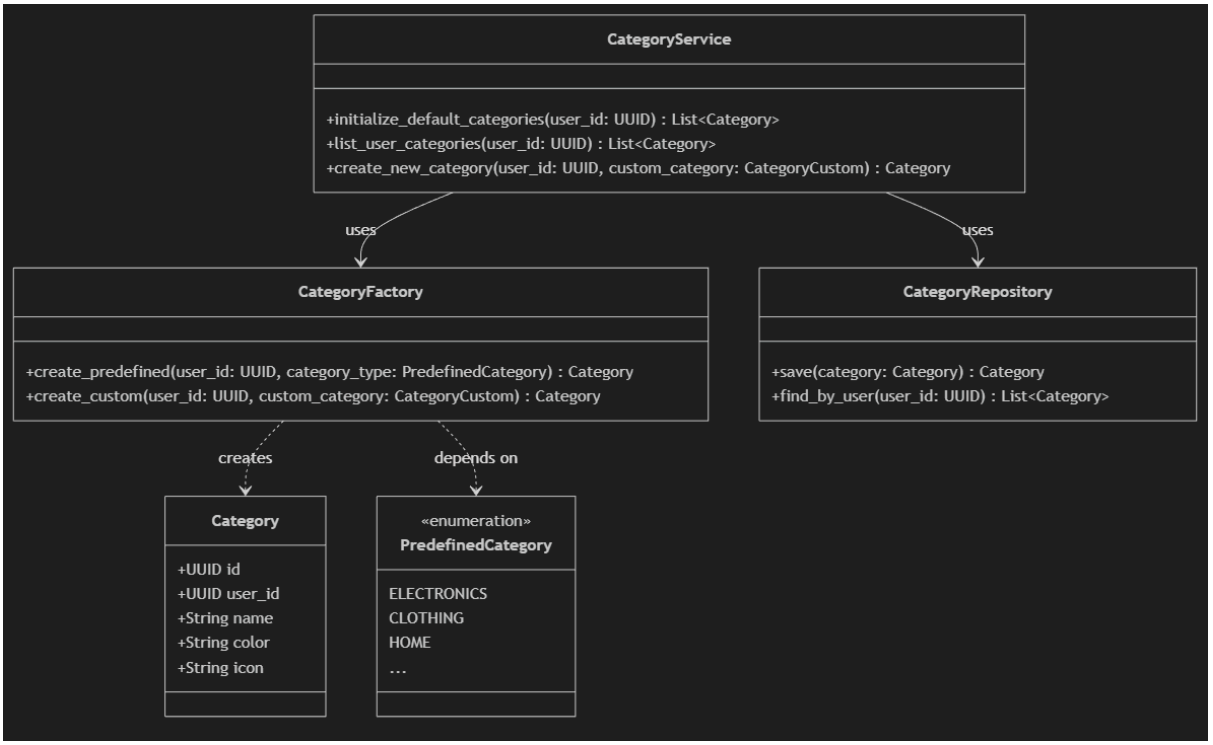


Figure 3. UC1: Class Diagram

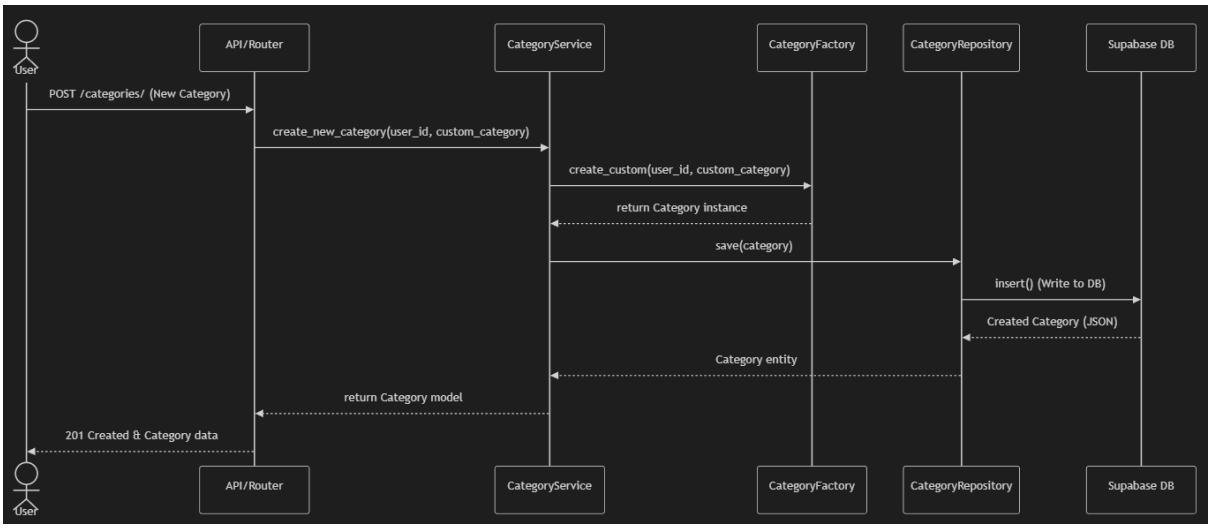


Figure 4. UC1: Sequence Diagram

2. Product Addition and Management

The process where users add a product to their lists by providing a URL. Under the hood, ScraperFactory determines the appropriate Scraper strategy (like AmazonScraper, etc.) based on the domain URL and extracts the product information.

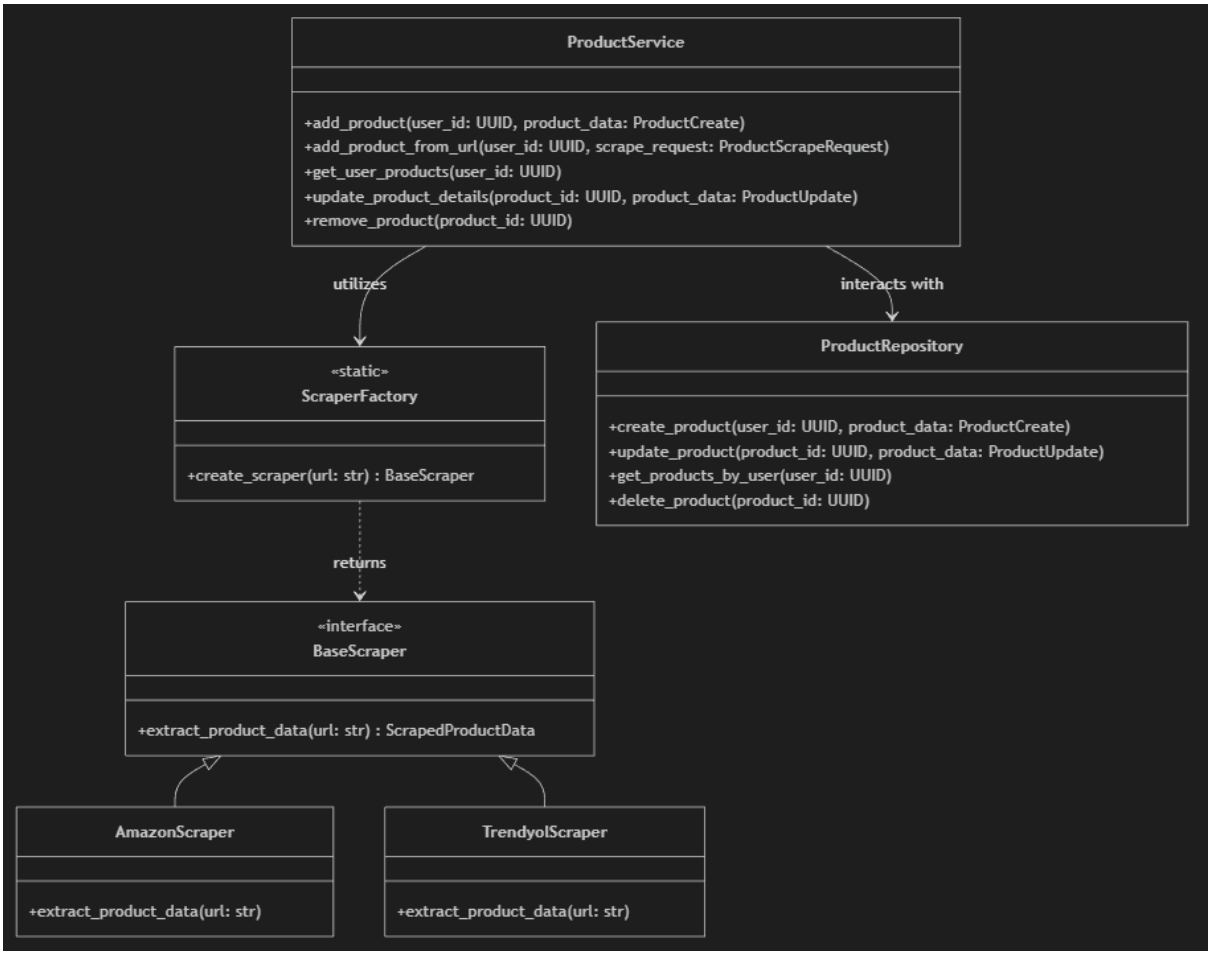


Figure 5. UC2: Class Diagram

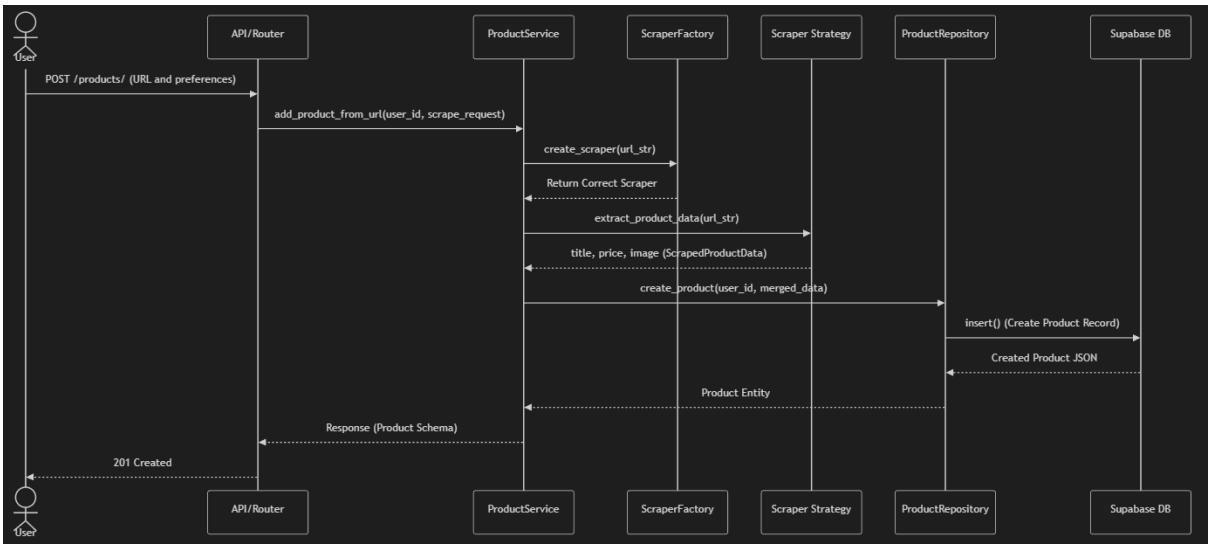


Figure 6. UC2: Sequence Diagram

3. Automatic Price Monitoring

Automated process where prices of tracked items (auto_track: True) are checked periodically by the application's Scheduler. If a price change occurs, it is persisted to the database and reflected in the price history. When the updated price hits the threshold (target_price), a notification is dispatched utilizing the Observer pattern.

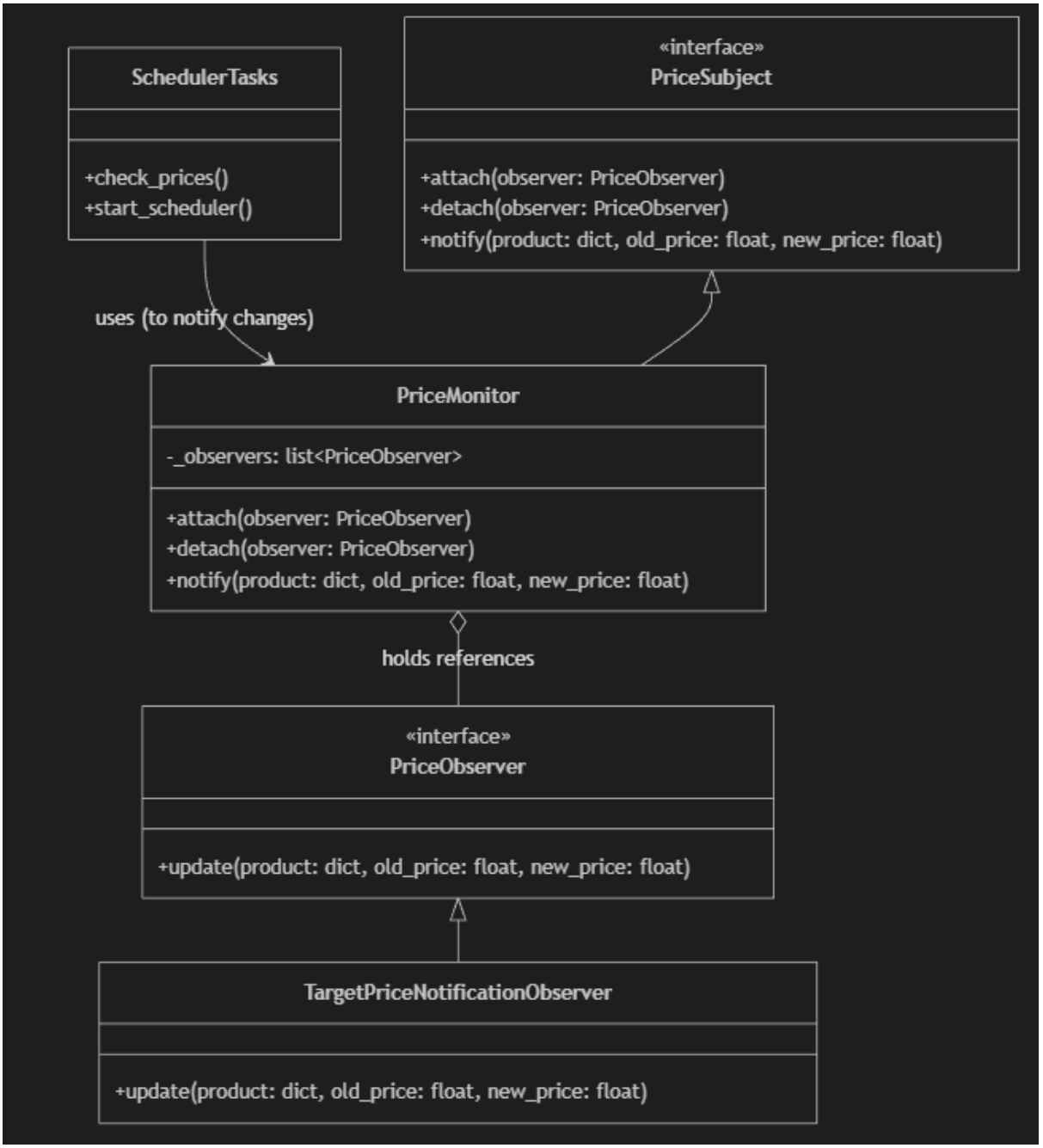


Figure 7. UC3: Class Diagram

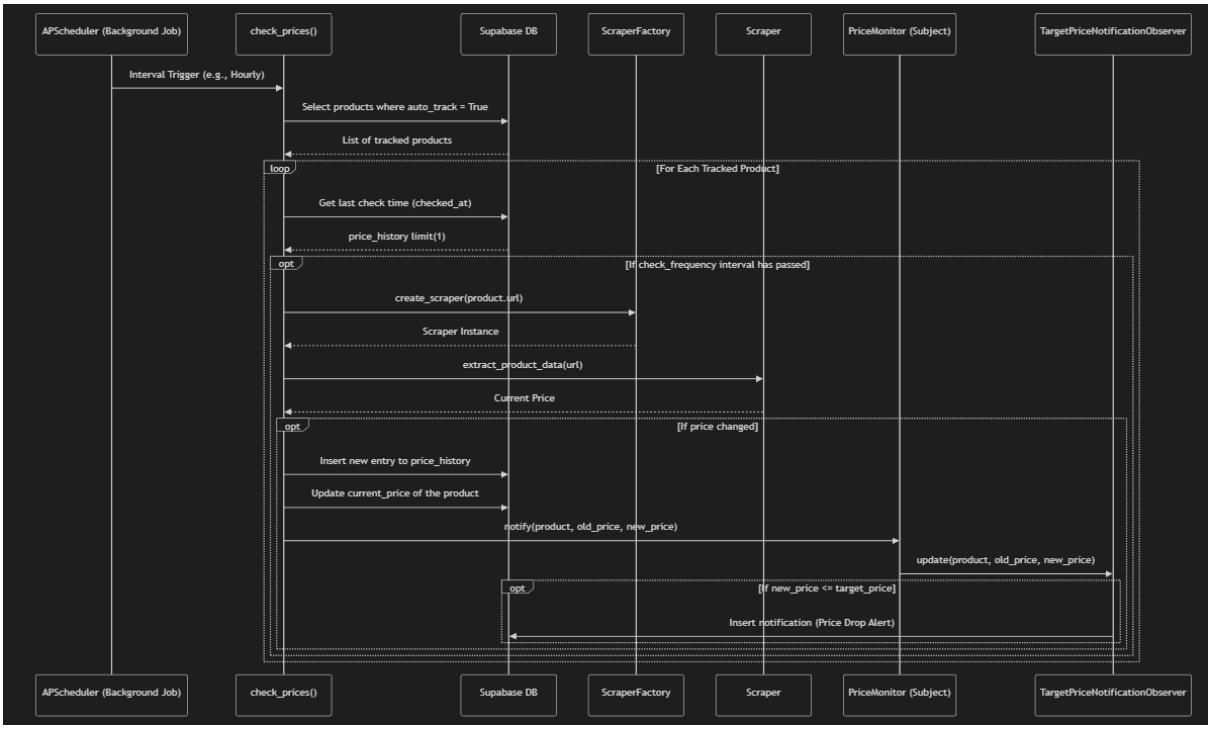


Figure 8. UC3: Sequence Diagram

4. Notification and Frequency Configuration

Users specify how often they want prices to be checked (check_frequency), whether they want automated tracking (auto_track), and the desired threshold (target_price). These configurations are set via the API tier and dictate the scraping intervals for the background scheduler operations.

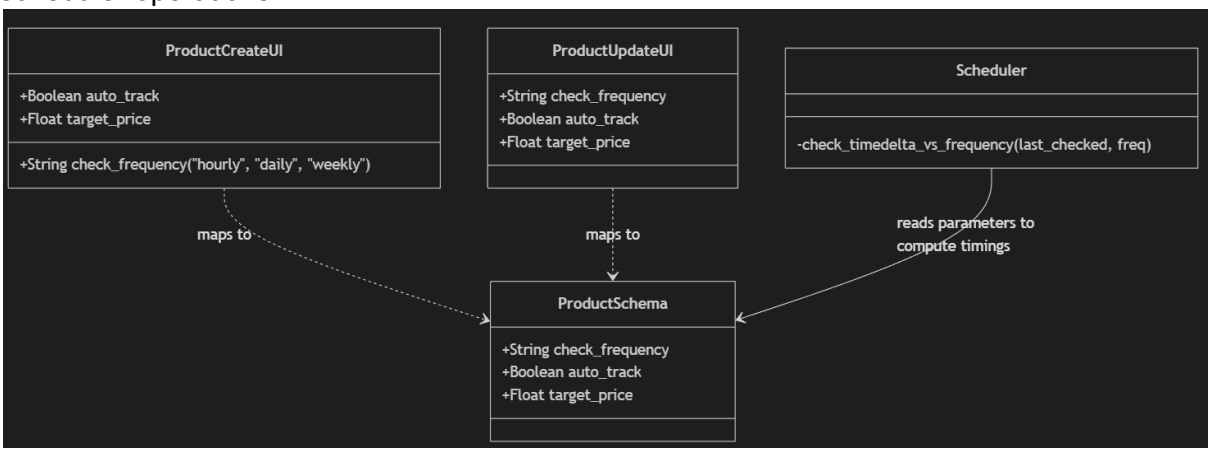


Figure 9. UC4: Class Diagram

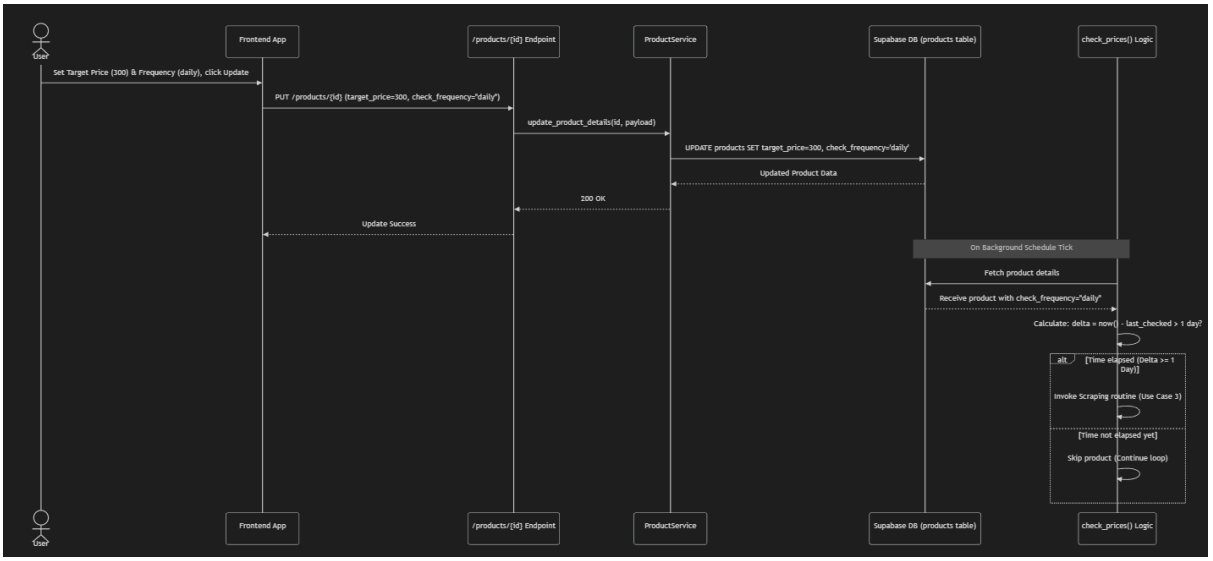


Figure 10. UC4: Sequence Diagram

7. Data Design

7.1 Data Model

User

Represents a platform user.
Attributes: id, first_name, last_name, email, created_at

Category

Logical buckets for products.
Attributes: id, name, category_type, user_id, created_at
Constraints: (user_id, category_type) unique

Product

A tracked URL representation.
Attributes: id, user_id, category_id, name, url, priority, check_frequency, auto_track, current_price, target_price, image_url, purchase_state ('pending', 'purchased'), is_favorite (boolean)
Constraints: Cascade on User delete

PriceHistory

Chronological sequence.
Attributes: id, product_id, price, checked_at

Notification

Alerts tied to users.
Attributes: id, user_id, product_id, old_price, new_price, is_read, type ('price_drop', 'target_reached')

7.2 Data Validations & RDB Constraints

- PostgreSQL Foreign Keys with ON DELETE CASCADE ensures deleting users handles orphans.
- Pydantic strictly validates numeric precision for prices and enums (hourly, daily, weekly).

8. Design Patterns

The architecture utilizes three prominent GoF design patterns to assure SOLID compliance.

8.1 Factory Pattern

Context: Creating Categories and picking Scrapers logic.
Application:

- CategoryFactory distinguishes constructing default categories (setting is_default) vs custom user inputs.
- ScraperFactory.create_scraper(url) parses a URL and yields the correct initialized scraper matching the host domain dynamically.

8.2 Strategy Pattern

Context: Abstracting the extraction algorithm.
Application: Every e-commerce site requires different HTML elements to be parsed. By using a base ScraperStrategy, the specific implementation (e.g., AmazonScraper, TrendyolScraper) is shielded. The system also introduces an AI-based LLMScraper as a fallback strategy when domain parsing fails manually.

8.3 Observer Pattern

Context: Decoupling side effects originating from Scheduler updates.
Application: Implemented via PriceObserver & TargetPriceNotificationObserver. When the scheduler.py detects a price change, it invokes price_monitor.notify(). The observer processes whether to generate a DB notification or drop it, decoupling notification mechanics from basic data writes.

9. Implementation Notes

9.1 Technology Stack

Layer	Technology / Tool
Frontend	React (JavaScript/JSX, Tailwind CSS)
Backend	FastAPI (Python)
Database & BaaS	Supabase (PostgreSQL)
Web Scraping	BeautifulSoup4, httpx, LLM integrations (Groq)
Task Scheduling	APScheduler (background price checks)
Version Control	Git
API Communication	REST over HTTPS, JSON payloads

9.2 Directory Layout

```
backend/
  app/
    api/              # FastAPI Router endpoints
    database/         # Migration scripts & auth client
    factories/        # Category, Scraper, User Factories
    models/ & schemas/# Pydantic Schema validations
    observers/        # Observer Pattern Event Handlers
    repositories/     # Data Access layer
    scheduler/        # Background execution (APScheduler)
    scrapers/         # Strategy algorithms for URLs
    services/         # Main logic
```

9.3 Backend Implementation Guidelines

9.3.1 Layered Architecture

The backend strictly follows a three-layer architecture. No layer should skip the one above or below it:

- Router layer: handles HTTP request/response only. No business logic.
- Service layer: contains all business rules. Calls repositories; never calls Supabase directly.
- Repository layer: contains all Supabase/PostgreSQL queries. Returns plain Python objects or dicts.

9.3.2 Pydantic Schemas

All incoming request bodies and outgoing responses are validated using Pydantic models. Validation errors are automatically converted to HTTP 422 responses by FastAPI.

9.3.3 Scraper and Strategy Implementation

All scraper classes reside in app/scrapers/. The ScraperFactory.create(url) method is the single entry point for obtaining a scraper instance. Scrapers must handle network errors and raise a ScrapingError. The scheduler catches ScrapingError, logs the failure, and skips updating the price for that cycle.

9.3.4 Background Scheduler

APScheduler runs inside the FastAPI process and triggers price-check jobs based on each product's configured frequency (hourly / daily / weekly). Jobs are stored in memory; on restart, active products are re-scheduled from the database. Job execution must not block the FastAPI event loop.

9.4 Frontend Implementation Guidelines

9.4.1 Component Structure

- Page components (src/pages/): correspond to routes; responsible for fetching data and passing it to child components.
- UI components (src/components/): stateless or locally-stateful; receive data via props; handle only presentation logic.

9.4.2 API Communication

All HTTP calls to the FastAPI backend are centralized in src/services/. Components never call fetch() or axios directly.

9.4.3 State Management

React Context is used for lightweight global state (authenticated user, category list). Component-level state (useState / useReducer) is preferred for local UI state.

9.4.4 Price History Chart

The price history visualization is implemented using a charting library (Recharts or Chart.js). Price history data is fetched as {timestamp, price} records and rendered as a line graph.

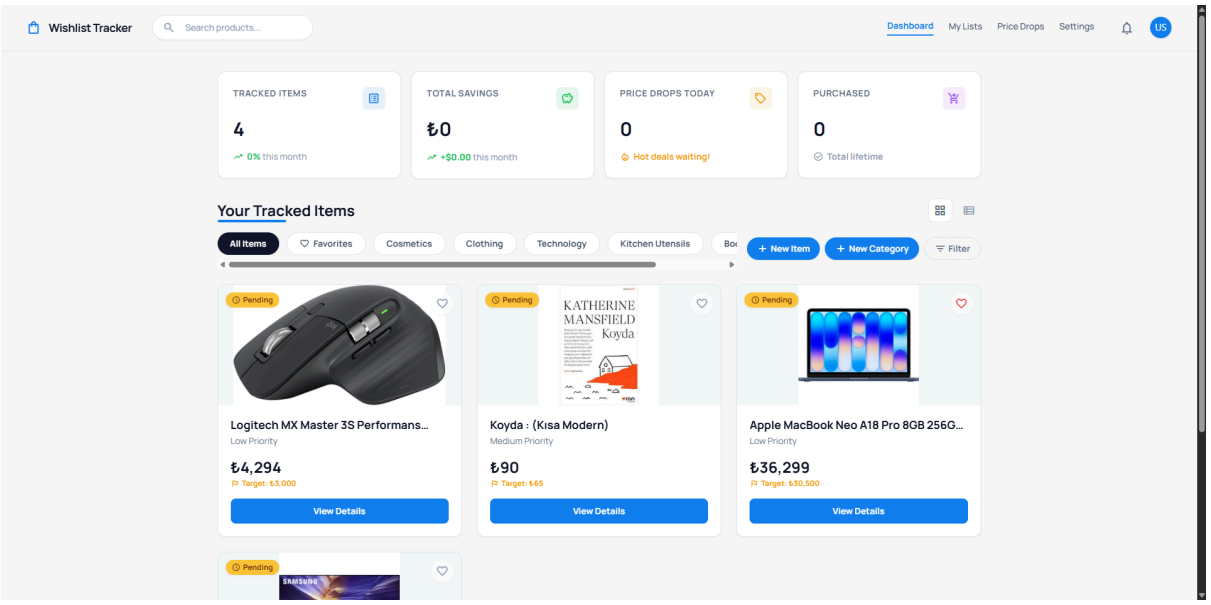
9.5 Known Limitations and Constraints

- User authentication is not yet fully implemented; user_id is currently passed manually via query parameters during development.
- Scraping availability cannot be guaranteed for all e-commerce websites due to anti-bot mechanisms and DOM structure changes.
- The scheduler runs in-process with FastAPI; for production scale, a dedicated task queue (e.g., Celery with Redis) would be preferable.
- Price checks are limited by Supabase free-tier rate limits and external website anti-scraping protections.

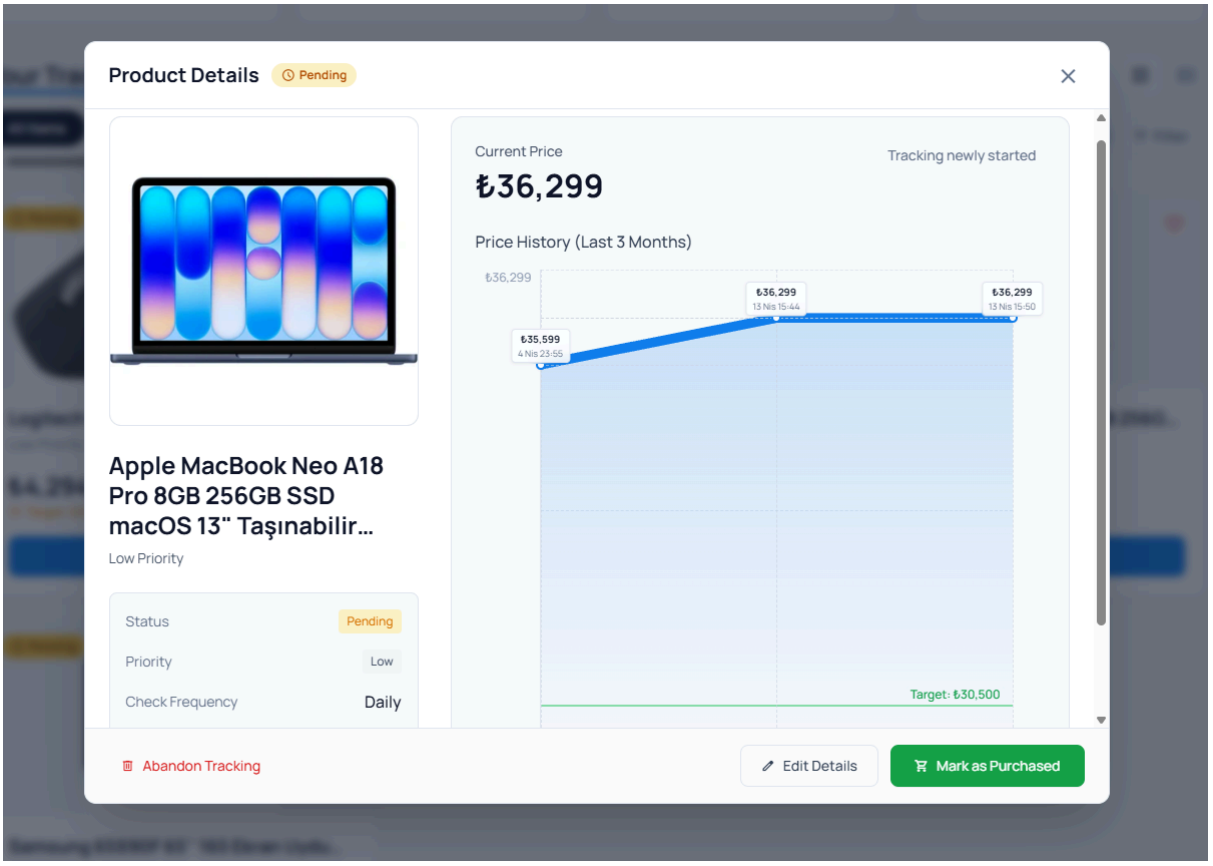
10. User Interface Design

The UI consists of the following key screens:

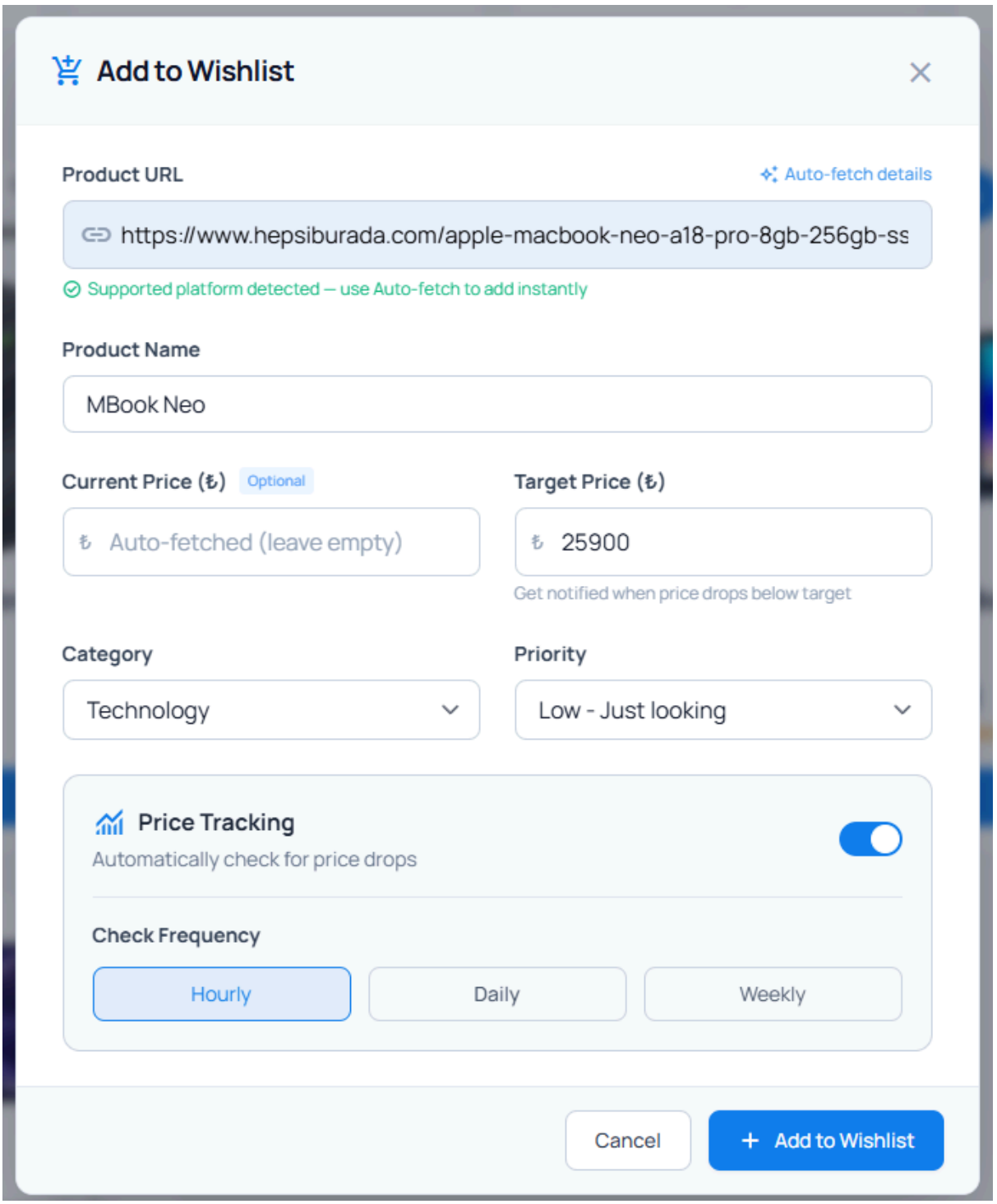
- Dashboard: A dense grid containing filter pills (Categories / Favorites), Summary Stats (Savings layout), and visually identifiable interactive cards per product.



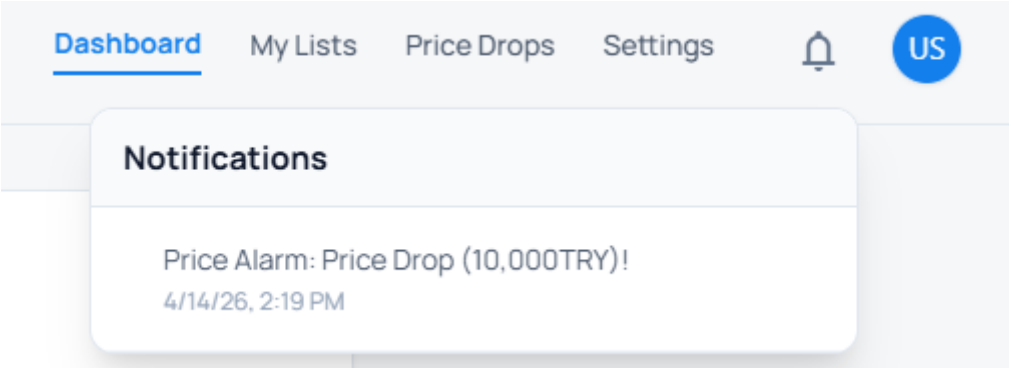
- Product Detail Modal: Full price history chart, metadata (category, priority, check frequency), and action buttons (purchase / edit / abandon).



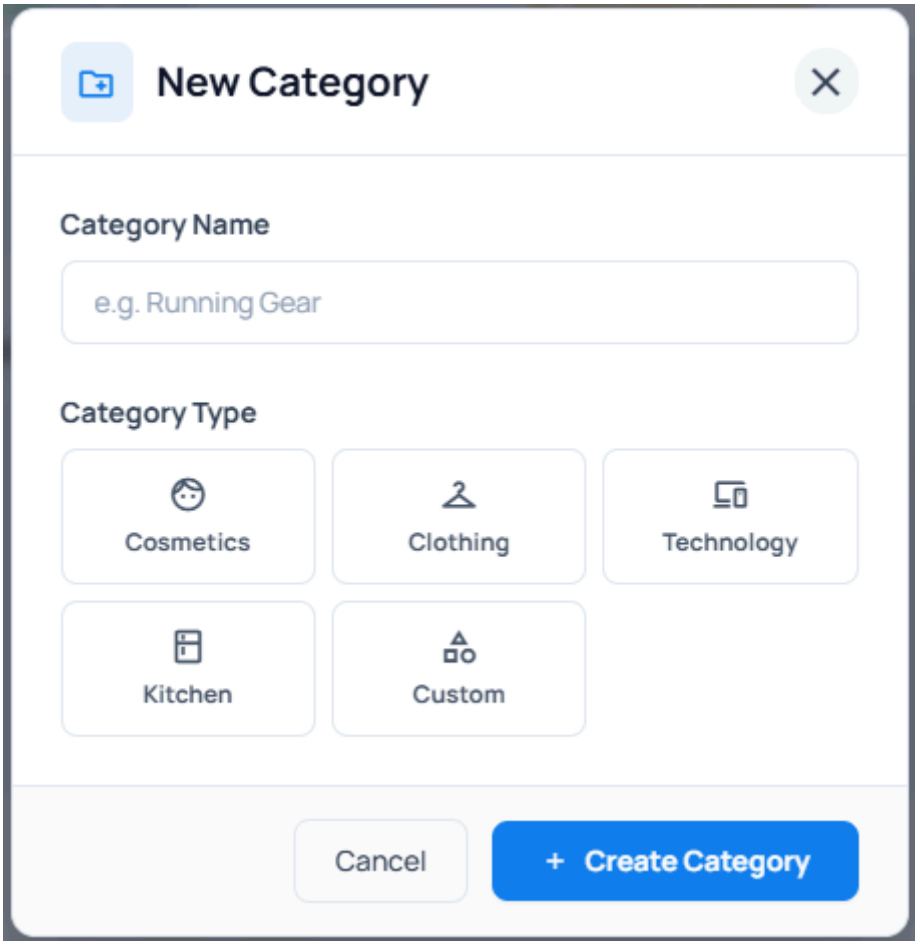
- Add Product Modal: Full form with URL input, category & priority dropdowns, frequency selector, and an auto-tracking toggle.



- Notifications: Unread/read states with product name, old -> new price, and timestamps.



- Categories: Visual category management grid.



11. External Interfaces

11.1 Key API Mappings

Endpoint	Description
GET /api/categories	List user categories
POST /api/categories	Create custom category
GET /api/products	List user products

POST /api/products/scrape	Initiates factory creation with URL scraping
PATCH /api/products/{id}	Update favorites, edit attributes
GET /api/notifications	List user notifications

11.2 Interop with Scraped Systems

The scraper subsystem strictly throttles requests and limits connections via the Strategy classes to minimize firewall blocking. A minimum 2-second delay is enforced between consecutive requests to the same domain, and HTTP timeouts are set to 10 seconds.

12. Performance Considerations

12.1 Performance Requirements

Requirement	Target	Notes
API response time	< 2s (p95)	Under normal single-user load
Price-check job execution	< 10s per product	Includes HTTP fetch + HTML parse + DB write
Frontend initial load	< 3s	Dashboard with up to 50 products
Notification delivery latency	< 30s	In-app notification; depends on scheduler interval
Database query time	< 500ms	All Supabase queries; enforced via indexed columns

12.2 Scalability and Optimization Strategies

12.2.1 Database Indexing

Critical columns indexed: products.user_id, price_history.product_id, and notifications.is_read are heavily indexed via migration logic.

12.2.2 Scheduler Design for Efficiency

The scheduler.py leverages IntervalTrigger combined with logical delta skipping (e.g., delta < timedelta(weeks=1) for 'weekly' checks), reducing system overloading and keeping Supabase traffic safely below free-tier constraints.

12.2.3 Scraping Request Throttling

- Minimum 2-second delay between consecutive requests to the same domain.
- HTTP connection timeouts set to 10 seconds.
- Failed requests are retried at the next scheduled interval, not immediately.

13. Error Handling and Logging

13.1 Exception Management

The system implements structured exception handling in the service and repository layers:

- DuplicateCategoryError: raised when a user attempts to create a duplicate category.
- CategoryNotFoundError: raised when a category ID does not exist during update or deletion.
- DatabaseError: raised when a Supabase operation fails.
- FastAPI automatically converts validation failures into HTTP 422 responses.

13.2 Logging Mechanisms

Logging is implemented using the structured logging library Loguru. The system records category creation, duplicate category attempts, database errors, and unexpected exceptions, including timestamps and relevant identifiers.

14. Design for Testability

- Layer isolation: Routers, services, factories, and repositories are implemented as independent components.
- Dependency injection: Repositories can be replaced with mock implementations during unit testing.
- Factory testing: The CategoryFactory can be unit-tested independently.
- API testing: FastAPI provides a TestClient for automated endpoint testing.

15. Deployment and Installation Design

15.1 Environment Configuration

Required environment variables: SUPABASE_URL and SUPABASE_KEY, stored in the .env file and loaded using python-dotenv.

15.2 Packaging and Dependencies

Backend dependencies are defined in requirements.txt. Start the backend server with:

```
uvicorn app.main:app --reload
```

Start the frontend with:

```
npm install
npm start
```

17. Project Plan

17.1 Product Backlog

- PB1 - User can create a custom category
- PB2 - System creates default categories for a new user
- PB3 - User can edit a category
- PB4 - User can delete a category
- PB5 - User can view categories
- PB6 - User can add a product
- PB7 - User can assign category to product
- PB8 - User can set priority for product
- PB9 - User can update product

- PB10 - System retrieves product price automatically
- PB11 - System stores price history
- PB12 - System updates prices periodically
- PB13 - User can set price check frequency
- PB14 - System sends notification when price changes
- PB15 - User can view notifications
- PB16 - System handles scraping errors
- PB17 - System logs system events

17.2 Sprint Plan

Sprint 1 (Week 1-2)

- Category management (PB1-PB5)
- Basic API endpoints
- Database setup

Sprint 2 (Week 3-4)

- Product management (PB6-PB9)
- UI for product operations

Sprint 3 (Week 5-6)

- Price monitoring system (PB10-PB12)
- Scraper implementation
- Scheduler setup

Sprint 4 (Week 7-8)

- Notification system (PB13-PB15)
- Error handling & logging (PB16-PB17)
- Testing and bug fixing

18. Future Work and Open Issues

User Authentication Integration

The system currently receives the user_id manually. Future versions will integrate authentication so that the user identity is extracted from access tokens.

Category Deletion

The current implementation does not contain the category deletion feature yet. It will be implemented along with all other improvements.

Bil482 - Software Design Document: Wi$h Li$t **1**
1. System Overview 2
2. System Context 2
3. Key Features and Functionality 2
4. Assumptions and Dependencies 2
5. Architectural Design 3

6. Component Design 4
7. Data Design 10
8. Design Patterns 11
9. Implementation Notes 11
10. User Interface Design 13
11. External Interfaces 16
12. Performance Considerations 16
13. Error Handling and Logging 17
14. Design for Testability 17
15. Deployment and Installation Design 18
17. Project Plan 18
18. Future Work and Open Issues 19
19. Task Matrix 20

19. Task Matrix

Tasks	Akif Emre Reis	Alaaddin Gürsoy	Elif Serra Öncü	Gizem Elif Bayar
System Overview			✓	✓
System Context	✓			✓
Key Features and Functionality		✓	✓	
Assumptions and Dependencies	✓			✓

Architectural Design	✓	✓	✓	
Component Design		✓		
Data Design	✓	✓		
Design Patterns	✓	✓	✓	✓
Implementation Notes			✓	✓
User Interface Design		✓		
External Interfaces			✓	✓

Performance Considerations		✓		✓
Error Handling and Logging	✓			✓
Design for Testability	✓			
Deployment and Installation Design		✓	✓	
Project Plan	✓	✓	✓	
Future Work and Open Issues	✓		✓	✓